CounselCopilot

Catching and Repairing Policy and Legal Conflicts in NDA Drafts with Applicability-Aware Retrieval and a Red-Team Repair Loop

LMA Group Project 2026 – Project Proposal

Team members (roll numbers):

2025201049 2025201044 2025201068 2025201038

Summary in three lines

Problem: legal consultants who draft NDAs change often and cannot learn hundreds of pages of company policy, so their drafts quietly *miss* required terms or break rules.
Idea: a drafting copilot that finds every company rule and US-law provision that *applies to this deal* and checks the draft against each one, citing both sides. It then *hardens* the draft: an opposing-counsel agent hunts for loopholes, a repairer writes minimal redlines, and a regression checker re-runs every rule so that no fix breaks another.
Test: on real NDAs with expert-labelled and planted defects, and with deal facts the system must read from the NDA, does applicability-aware retrieval catch more omissions with fewer false alarms than standard RAG and long context, at the recall of checking every rule but with fewer checks? And does the red-team loop leave fewer problems and introduce fewer new ones than one-shot repair or self-refine?

1 Domain, target user and problem

Domain: Legal. **Scope:** one contract type (NDAs), one jurisdiction (USA), three companies' policy packs.
Target user: a legal consultant (contract staff, outsourced counsel or junior associate) drafting NDAs for a company, with a senior lawyer who signs off.

Problem statement. Companies spread their NDA requirements across many documents: data-protection, IP and vendor policies, an approval matrix and past negotiated positions. The consultants who draft NDAs rotate often, and a new consultant cannot absorb all of this in a few days. Their typical mistake is not bad wording but a *missing* or non-compliant term, such as no data-return clause, a confidentiality period shorter than policy allows, or a foreign governing law that needs General Counsel approval. Some drafts also contain terms that are risky under US law, such as a non-compete under California law (void under Cal. B&P Code §16600). These misses surface later as disputes. A reviewer that reads the draft once cannot find a missing clause, because there is nothing in the text to point at. We want a copilot that checks every applicable rule, then attacks and repairs the draft the way an experienced opposing lawyer and a senior colleague would, acting as knowledge transfer at the moment the consultant needs it.

Why it matters. NDAs are the most frequently signed contracts, and one missed rule can expose a company's data or IP. A senior lawyer can verify a cited flag in seconds but cannot re-read every policy for every draft.

2 What is missing in existing approaches

- **Contract-error benchmarks** (CLAUSE, ContractScrub, 2026) test prompted models on errors inside one contract (best macro recall 0.75); **ContractEval** (NLLP 2025) finds open models wrongly answer "no such clause" in 25–30% of cases. None checks a draft against a *company's own policies*.
- **PolicyGuard** (arXiv 2606.32004, 2026), the closest work, compiles one company's NDA playbook into logic rules and checks every rule. It is evaluated on 5 confidential contracts that cannot be released and ignores deal-specific applicability. Most importantly, it *only flags*: it writes no redlines, and its authors list cross-rule consistency as an open limitation.
- **Industry tools** (Harvey, LegalOn, Spellbook) offer "playbook review" with private evaluations only. **No open benchmark tests NDA policy compliance with omissions.**
- **Adversarial agents.** LegalSim and AgentCourt pit lawyer agents against each other in *litigation*,

not drafting; Self-Refine (2023) iterates without an adversary or a regression check.

3 Research questions and hypotheses

RQ1 (main). Does *applicability-aware two-way retrieval* catch more policy omissions and violations, with fewer false alarms, than (a) standard clause-to-policy RAG, (b) putting the whole policy pack into a long-context model, and (c) checking every rule exhaustively?

H1. With deal facts left off the form (so applicability must be inferred from the NDA), it raises omission recall by $\geq$**10 points** over long context and over naive RAG given a checklist of every rule, at equal or fewer false alarms per NDA. It is **equivalent within 3 points of recall** to exhaustive per-rule checking with the same deal filter while making fewer checks, and its advantage over long context **grows with policy size**.

RQ2. Does a *counterexample-guided red-team and repair loop* leave fewer violations, omissions and loopholes, and introduce fewer new conflicts, than one-shot repair or self-refine with the same model and token budget?

H2. Remaining problems per NDA are **at most half** those after one-shot repair, also when one-shot repair is given the attacker's scenarios. Fewer than **5%** of redlines introduce a new violation, and $\geq$**70%** of redlines are accepted in a blind review of 50 NDAs by two teammates who did not build the loop.

4 Proposed system

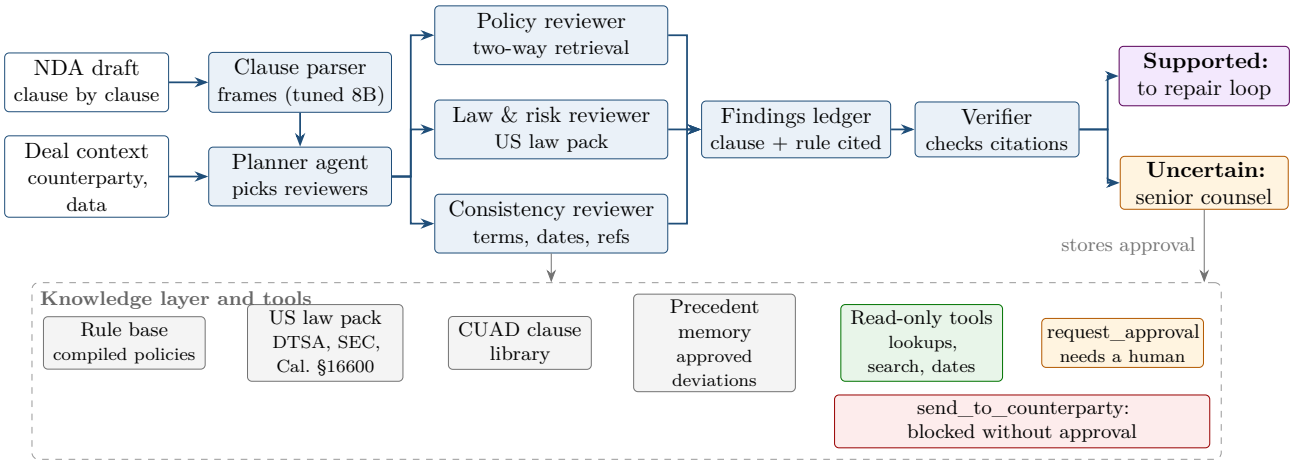


Figure 1: Detection pipeline (RQ1). Blue boxes are LLM agents (same backbone in every system we compare). Green tools are read-only, orange needs human approval, and red is never allowed without approval. Supported findings go on to the red-team and repair loop in Figure 3.

How it works. The *clause parser* turns each clause into an *obligation frame* {party, action, object, condition, duration, exception}, so numbers and conditions can be compared exactly rather than by text similarity. Offline, a *policy compiler* turns each company's prose policies into a rule base, where every rule has an ID, an applicability condition (e.g. `vendor AND personal_data`), a requirement and its source text. The *planner* picks reviewers based on the deal and the clause types present. Three reviewers call tools and write findings to a shared ledger. The *verifier* keeps a finding only if both citations support it, and sends uncertain or approval-gated cases to a senior lawyer; approved deviations go into *precedent memory*.

RAG design. Retrieval combines BM25 and dense search with a cross-encoder reranker, so legal terms like "indemnify" match exactly. Policies are chunked by rule rather than by fixed size, so an exception stays with its rule. A one-hop cross-reference graph pulls in linked approval rules and statutory exceptions. Because Gemini can read a whole 40k-token pack, we do not assume RAG is needed. We *measure* it with a policy-size sweep at 40k and 150k tokens (400k if time allows), padding with real public policies and recompiling the rule base at each size.

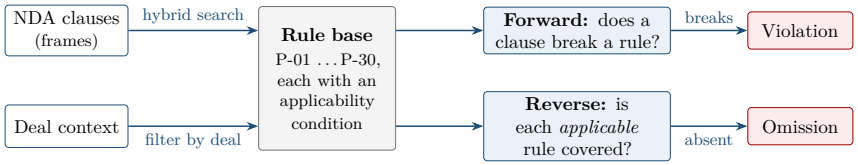


Figure 2: Two-way retrieval. The reverse path is the only way to find a *missing* clause, and the deal filter stops rules that don't apply to this deal from firing.

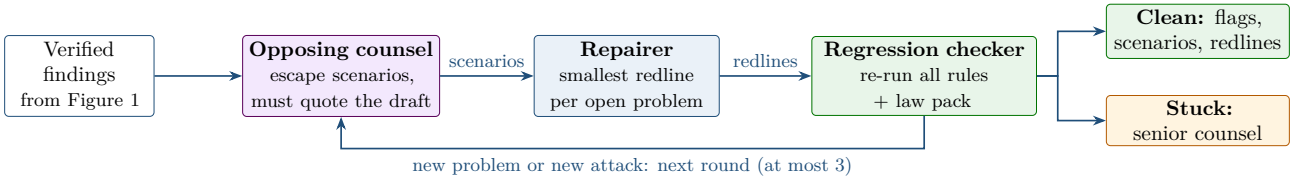


Figure 3: Red-team and repair loop (RQ2), borrowed from counterexample-guided refinement (formal verification) and regression testing (program repair). At most 3 rounds. Final scores use gold original clauses, held-out checks and a validated judge, never this loop's own checker.

Red-team and repair loop. Fixes interact (lengthening a term for one rule can break another rule's maximum), and checklists miss what the other side could *exploit*, such as a residuals clause. So after detection, an *opposing-counsel agent* writes escape scenarios that must quote the draft and pass the verifier. A *repairer* writes the smallest clause-level redline for each open problem, and a *regression checker* re-runs the whole rule base and law pack on the edited draft. The loop stops when nothing new is found, or escalates when the budget runs out or two rules can't both be met. For planted defects the ground truth is exact: each was made by editing an original clause, so *the original is the gold repair*. ContractNLI's expert labels mark natural loopholes (e.g. oral disclosures not covered). Everything else in a repaired draft is scored by an independent judge, validated against expert labels, with fixes and regressions confirmed by a second run.

5 The five capabilities: why each is needed and how we measure it

Capability	Why this problem needs it	How we measure it
Fine-tuning	One QLoRA Llama-3.1-8B for clause tags (CUAD), ContractNLI checks and obligation frames. The regression checker re-runs every rule after every repair round, so checks must be cheap, local and consistent.	CUAD tag F1; ContractNLI acc./macro-F1 (12 trained, 5 held-out checks); frame field accuracy on 300 gold frames; tuned vs untuned vs prompted Gemini
RAG	An omission check must find *every* applicable rule, cite it exactly and stay cheap as the policy corpus grows.	Applicable-rule recall@k, omission recall, nDCG@10, policy-size sweep vs long context
Reasoning	Deciding whether a rule applies to this deal; reasoning about how a clause can be exploited; planning repairs under interacting rules; verification.	Accuracy of deal facts inferred from the NDA, and of applicability decisions, vs gold; loophole recall; regression rate; verifier precision; cause of each miss
Agentic	Planner and reviewers; an **attacker vs repairer loop** with a stopping rule; shared ledger, precedent memory, hand-off to a senior lawyer.	Loop vs one-shot repair vs self-refine; rounds to convergence; planner vs fixed pipeline; hand-off precision/recall
Tool calling	Exact bookkeeping (terms, dates, cross-references), rule/statute/approval lookups, applying redlines, full re-checks, gated actions.	Tool precision/recall vs gold tool sets per defect type, argument validity, recovery from 10% injected errors, **attempted unsafe action rate (target 0)**

6 Dataset and evaluation environment: PolicyNDA

- **NDAs:** ContractNLI (607 real NDAs, split 423/61/123, CC BY 4.0). **Clause library:** CUAD (510 contracts, 41 clause types).
- **Company policy packs (generated, then reviewed):** 3 fictional US companies, each with ∼30 NDA rules buried in ∼40k tokens of realistic prose. Claude (Sonnet), a model family that no system

under test, compiler or judge uses, writes the rules first and the prose second; Claude Haiku checks that the prose states each rule, and a teammate reviews flagged rules, every anchor and a random sample. Gold rule lists are kept separate.

- **US law pack** (of the 57 test NDAs that name a governing law, about 37 name a US state): 7 clear-cut provisions (DTSA notice, SEC Rule 21F-17, Speak Out Act, Cal. §16600 for non-competes and non-solicits, penalties, CCPA), each checked by two teammates against the official text.
- **Ground truth without circularity:** about 60% of the rules are anchored to ContractNLI's 17 expert-labelled checks (e.g. "vendor NDAs must require return of information" ↔ *return of confidential information*). So each real NDA's *natural* omissions and violations are already labelled by legal experts. Anchored rules carry no numbers. Field rules (governing law, term length) and the other rules are labelled independently by two Claude models (95% of label keys agree); a person settles every disagreement and checks a 20% sample of agreements, and label accuracy is reported. Unclear labels are left unscored, never counted as omissions.
- **Planted defects on top:** violations, omissions, approval-gated deviations, statutory risks and *distractors* (rules that look relevant but don't apply), giving 2–4 per NDA; a planted edit may not break any other rule. About 25% of NDAs get only distractors, so we can measure false alarms. 20 NDAs also get prompt-injection text, which changes no gold.
- **Loopholes and gold repairs (RQ2):** natural loopholes come from three ContractNLI checks (technical-only definition, oral disclosures excluded, retained copies; standard carve-outs are not counted), and one more is planted in half of the test NDAs (residuals clauses, carve-outs with no time limit). Gold repairs are the original clauses behind planted defects, plus gold frames for natural defects on 40 NDAs.

7 Evaluation plan

Two-stage backbone. Stage 1 runs everything on **Llama-3.1-8B** served free on Kaggle GPUs. Stage 2 confirms the main systems on **Gemini 3.8 Flash**. All systems in a comparison use the same backbone. Long-context runs and the policy-size sweep need Gemini, because a T4 GPU cannot hold a 40k-token prompt.

RQ1 baselines	B1 long context (whole pack in one prompt) · B2 naive RAG (fixed chunks, forward only) · B2+checklist (naive RAG plus a check of every rule) · B3 single agent with tools · B4 exhaustive per-rule NLI checking, with and without the deal filter
RQ2 baselines	R1 one-shot "fix all issues" repair · R1+A one-shot repair given the attacker's scenarios · R2 Self-Refine (no attacker, no full re-check). All start from the same findings with the same token budget, checker tokens included.
Ablations (one change each)	no reverse retrieval (clean RQ1 test) · no deal filter · no frames · no verifier · planner → fixed pipeline · untuned model · no cross-reference graph · no memory · loop: no attacker / no regression check / 1 vs 3 rounds / attacker on a different model · **local-first cascade** (Gemini only for unsure or high-severity checks) vs all-Gemini
Detection	Recall, precision and F1 per defect type; **omission recall**; false alarms per NDA. A finding counts only if both citations are correct (a rule-only score is also reported). Natural and planted defects are reported separately.
Repair loop	Remaining defects and loopholes per NDA; **regression rate** (share of redlines that break a rule); repair success vs gold; attacker loophole recall and precision; edit size; rounds; blind acceptance (two raters, κ)
Cost	$ per NDA; share of checks escalated; misses by severity vs escalation rate
Safety	Attempted unsafe/unapproved action rate (target 0) and change in recall on the 20 NDAs with prompt-injection text; hand-off precision and recall
Rigour	Paired bootstrap 95% CIs over NDAs; an equivalence test (TOST) for the 3-point margin; 3 deal draws per NDA. The RQ2 judge is validated against ContractNLI expert labels and 100 human labels (Cohen's κ), and its noise floor is measured by judging unchanged drafts. Five worked qualitative cases and a table of misses by cause.

8 Expected novelty

1. **PolicyNDA**, the first *open* benchmark for checking NDAs against company policy. It covers omissions and deal-specific applicability, with ground truth anchored to expert labels.

2. **Applicability-aware two-way retrieval** with deal facts read from the NDA, compared against exhaustive per-rule checking and long context as the corpus grows.
3. **A counterexample-guided red-team and repair loop** for contract drafting. An opposing-counsel agent attacks, a repairer fixes and a regression checker guards every edit. This adds the repair, adversary and cross-rule checking that PolicyGuard lacks, measured against exact gold repairs.

9 Safety plan

- Only public NDAs and synthetic policies, so no client or personal data. Llama runs locally; only public or synthetic text goes to the Gemini API, on the paid tier.
- Gated actions: `request_approval` needs human confirmation, and `send_to_counterparty` is a mock that is blocked without approval. Misuse is measured, including under prompt injection.
- Redlines are suggestions; nothing is sent or signed. The output is decision support with citations, not legal advice: statutory flags read "risk under §16600", and the law pack is small, cited and dated.

10 Compute plan

Item	Resource	Estimate
QLoRA Llama-3.1-8B, multi-task, ∼18k examples, 2 runs	Kaggle T4 (free)	≈16 GPU-h
Tuned-model inference (ContractNLI test, regression checks in the loop)	Kaggle T4, vLLM	≈8 GPU-h
Stage 1: all systems, the loop and ablations on Llama-8B, 123 NDAs, 3 deal draws	Kaggle 2×T4, vLLM	≈35 GPU-h, $0
Stage 2: Full, B1, B2, B4 and 2 RQ1 ablations, 123 NDAs (≈18M in / 4M out, context-cached)	Gemini 3.8 Flash, Batch API	≈$15
RQ2 loop, R1, R2; policy-size sweep; local-first loop and ablation checks on 40 NDAs	Gemini 3.8 Flash, Batch API	≈$22
Silver frames (Flash); policy compilation (Pro, once); RQ2 judge on 40 NDAs (Pro)	Gemini, Batch API	≈$22
Policy packs and labels (Claude Sonnet and Haiku agents, no API)	Claude	$0
Total		**≈60 GPU-h, ≈$60**

Gemini prices checked 2 Oct 2026 (3.8 Flash $0.75 in / $3.75 out per M tokens until 31 Dec 2026, then double: ≈$110 in all if stage 2 runs later). 60 GPU-hours is about two weeks of Kaggle's free quota.

11 Risks and fallback plan

Risk	Fallback
Scope too large for the time	RQ1 and the loop with its regression checker are the core. Drop the different-model attacker first, then cap the loop at 2 rounds.
Loop games its own checker or invents loopholes	Scored only by gold originals, held-out checks and a validated judge; scenarios must quote the draft.
Llama-8B too weak for tool use	Constrained JSON decoding; hosted Llama-70B for stage 1.
Generated data favours one model	Packs and labels come from a family no tested system uses.
Gains come from memorised ContractNLI labels	5 checks are held out of fine-tuning and reported separately.
A baseline or self-refine matches our system	Report it; the benchmark, omission results, cost gap and regression checker remain contributions.
Unrealistic policies or law errors	Generator prompted with real policy structure; read-through by a teammate; two-person law check.

Key references. Koreeda & Manning, ContractNLI (2021) · Hendrycks et al., CUAD (2021) · CLAUSE (arXiv:2511.00340) · ContractScrub (arXiv:2608.20204) · ContractEval (arXiv:2508.03080) · PolicyGuard (arXiv:2606.32004) · CEGAR (Clarke et al. 2000) · Madaan et al., Self-Refine (2023) · LegalSim (arXiv:2510.03405) · AgentCourt (arXiv:2408.08089) · QLoRA (2023).